



FACULTAD
DE INGENIERÍA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Informe Trabajo Especial 2

Reproductor WAV con Ecualización

Alumno:
Federico Castiñeira

Profesores:
Federico Guerrero
Marcelo Haberman

14 de agosto de 2025

1. Descripción general del sistema

Se implementa un reproductor de archivos de audio .wav con filtros de ecualización y visualización del espectro ecualizado utilizando FreeRTOS. Se han generado 4 tareas, cada una con un propósito específico y fundamental para el funcionamiento del sistema. Se han utilizado elementos de sincronización entre tareas, como semáforos, notificaciones y colas, para lograr una buena coordinación. Por último se ha hecho uso de botones, un potenciómetro y una pantalla OLED, para el manejo de la interfaz, ecualización y visualización del sistema.

1.1. Esquema

En la Figura 1 se visualiza un diagrama esquemático de alto nivel del sistema a implementar.

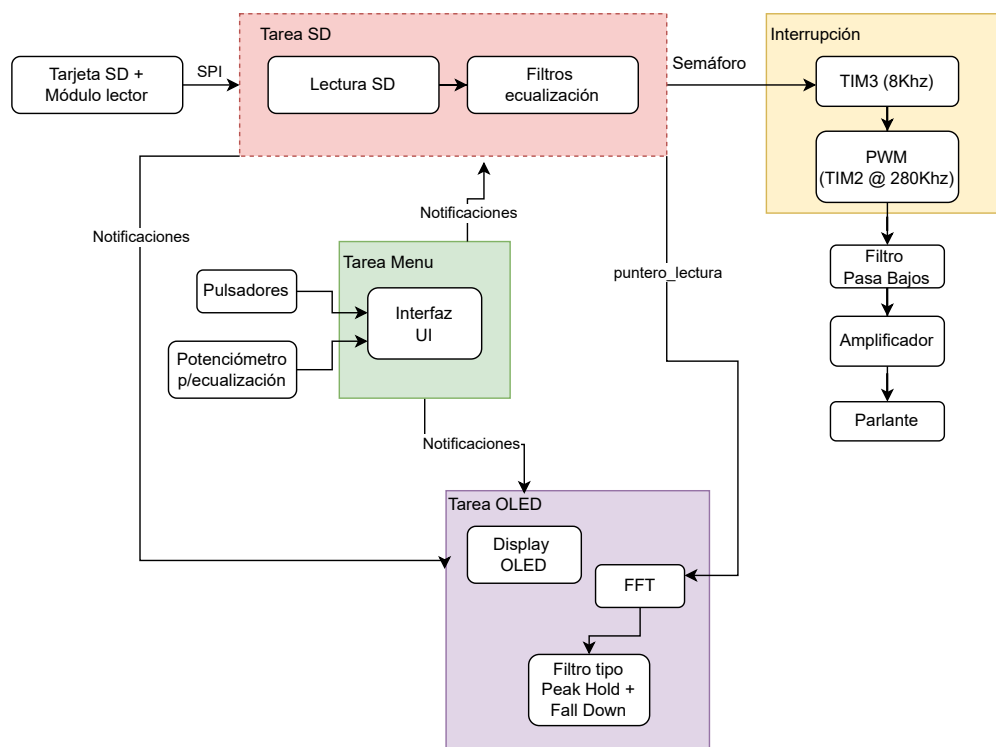


Figura 1: Diagrama esquemático del sistema

2. Tareas e Interrupciones

2.1. tareaSD

- **Prioridad:** High
- **Función:** Lee bloques de 512 bytes desde el archivo .wav, aplica filtro Biquad, rellena buffer ping-pong de audio y lo envía a la PWM. A su vez envía del mismo bloque de 512 bytes, 64 muestras a la cola para el cálculo de la FFT. A su vez implementa una maquina de estados finita para mejorar la eficiencia y comprensión del sistema. Utiliza 4 estados posibles: *INICIO*, *ESCUCHANDO*, *STOP* y *FIN*. Cada pasaje de estado lo maneja en concordancia con la *tareaUI* y con flags para determinar si lee desde la SD, reproduce la música, detiene la reproducción o termina y cierra el archivo.
- **Sincronización:** Espera la liberación de un **semáforo** por la interrupción del Timer 3 para la lectura de un nuevo bloque de bytes. Recibe **notificaciones** desde la *tareaUI* para definir el estado de la FSM y ejecutar las funciones.

2.2. tareaUI

- **Prioridad:** Baja
- **Función:** Permite la navegación por el menú de canciones mediante botones, definiendo los modos de las FSM de *tareaSD* y *tareaOLED*.
- **Sincronización:** Envía **notificaciones** a las demás tareas según el botón que se presione.

2.3. tareaOLED

- **Prioridad:** Baja
- **Función:** Mediante una máquina de estados finita determina qué visualizar en la pantalla OLED: *MENU*, *REPRODUCCIÓN* o *PAUSA*. A su vez cuando esta en el estado de *REPRODUCCIÓN*, calcula y grafica la FFT, leyendo 64 muestras desde el *puntero_lectura*. Utiliza funciones definidas en la librería *fft.h*.
- **Sincronización:** Espera **notificaciones** desde *tareaUI* para cambiar de estado y actualizar la pantalla.

2.4. Interrupción TIM3

- **Frecuencia:** 8 kHz
- **Acciones:**
 1. Toma muestra del buffer ping pong.
 2. La envía a TIM2 → CCR1 para actualizar el duty cycle de la PWM.
 3. Libera semáforo para que la tarea LeerSD cargue nuevas muestras.

2.5. PWM TIM2

- **Frecuencia:** 280 kHz (resolución de 8 bits con $ARR = 255$).
- **Salida:** conectada a filtro pasa bajos + amplificador de audio. Se ha optado por una resistencia de $4.7k\Omega$ y un capacitor de 10 nF, de tal manera de lograr un frecuencia de corte que ronde por los 4kHz.

3. Uso de memoria

El siguiente cuadro resume el stack reservado por cada tarea del sistema operativo FreeRTOS:

Tarea	Stack size (words)	RAM utilizada
tareaSD	256	1024 B
tareaUI	128	512 B
tareaOLED	256	800 B

Cuadro 1: Tamaños de stack de cada tarea

3.1. Heap del sistema operativo

- **Configurado:** 5000 B.
- **Heap libre:** ~1488 B.

3.2. Resumen de uso de memoria

A modo de mejora, se ha utilizado como ajuste de compilación, *OS-SIZE*, para que optimice por tamaño y mejorar la eficiencia del uso de la memoria.

- **RAM utilizada:** 95.31 % (~960 B de 20 KB)
- **FLASH utilizada:** 68.89 % (~19.91 KB de 64 KB)

Depuración del stack en compilación

Mediante el *Static Stack Analyzer* se ha visualizado el máximo costo de bytes de cada tarea, considerando el peor de los casos, obteniendo lo siguiente:

Tarea	Max Cost (bytes)
entryLeerSD	800
entryUI	112
entryOLED	432

Cuadro 2: Máximo costo en bytes de las tareas

Se verifica que la asignación previa del stack ha sido suficiente y a su vez conservadora ya que hay stack sobrante.

Depuración del stack en ejecución

A partir de las funciones, dispuestas desde FreeRTOS, *osThreadGetStackSpace(taskHandle)* y *xPortGetFreeHeapSize()* se ha verificado durante ejecución del programa, el stack libre de cada tarea como también del heap. Desde la tarea default que provee FreeRTOS, se han ejecutado dichas funciones, resultando lo siguiente:

Tareas y Heap	Usado (bytes)	libre (bytes)
entryLeerSD	504	520
entryUI	112	400
entryOLED	456	568
Heap	3808	1192

Cuadro 3: costo en bytes de las tareas

4. Validación temporal y depuración

4.1. Capturas del analizador lógico

Se han utilizado *Trace Hooks* para realizar el seguimiento de la ejecución de las tareas, de la interrupción y de la señal PWM. Luego, con el programa Logic Analyzer, se han llevado a cabo las correspondientes mediciones para verificar el comportamiento correcto de cada evento y, a su vez, obtener el factor de utilización del sistema. Se capturaron las siguientes señales digitales:

- tareaSD (Canal 0)
- tareaOLED (Canal 1)
- tareaUI (Canal 2)
- tareaDefault (Canal 3)
- tareaIDLE (Canal 4)

- Pulso generado por interrupción del TIM3 (Canal 5)
- Salida PWM del TIM2 (Canal 6)

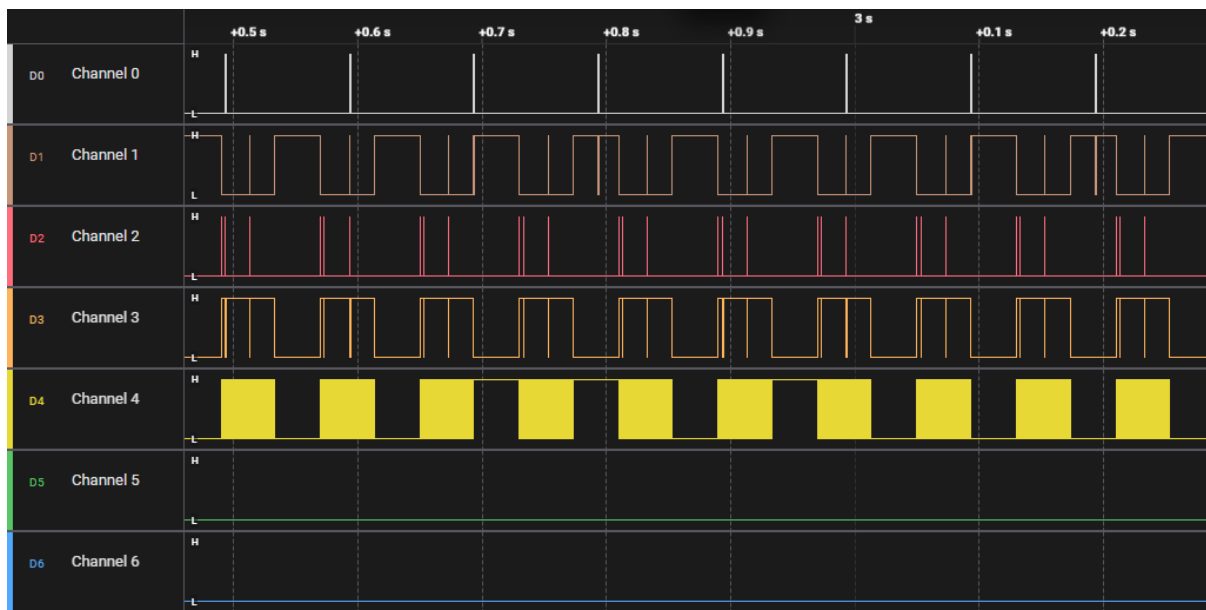


Figura 2: Capturas en No reproducción

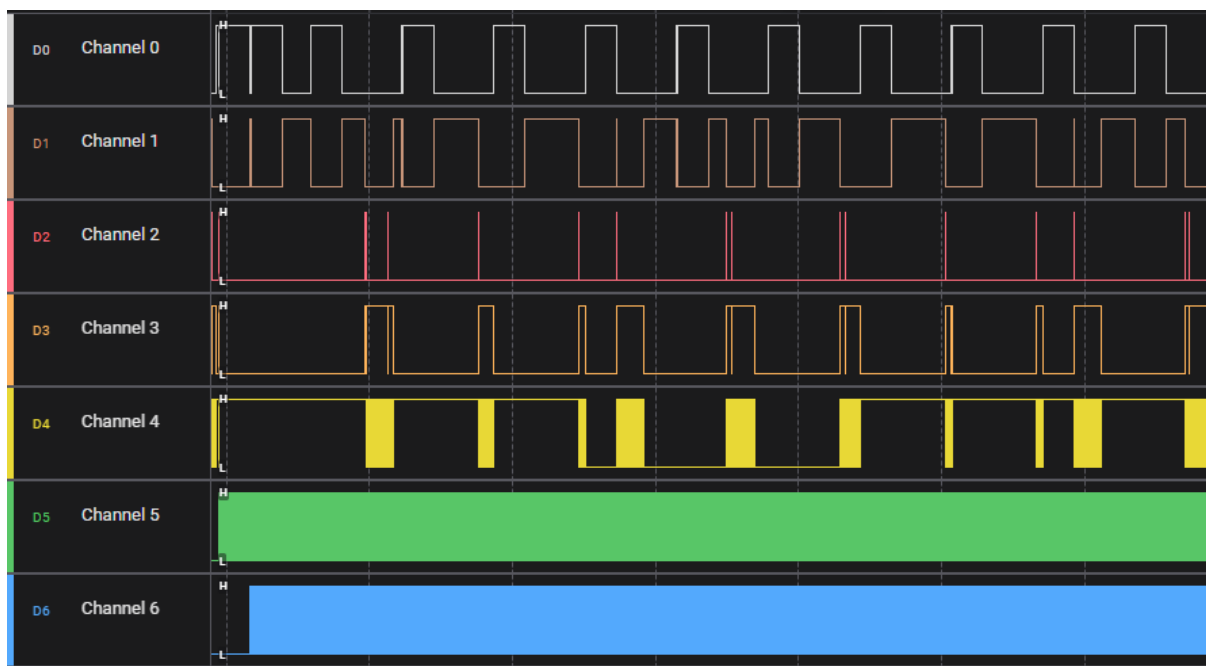


Figura 3: Capturas en reproducción

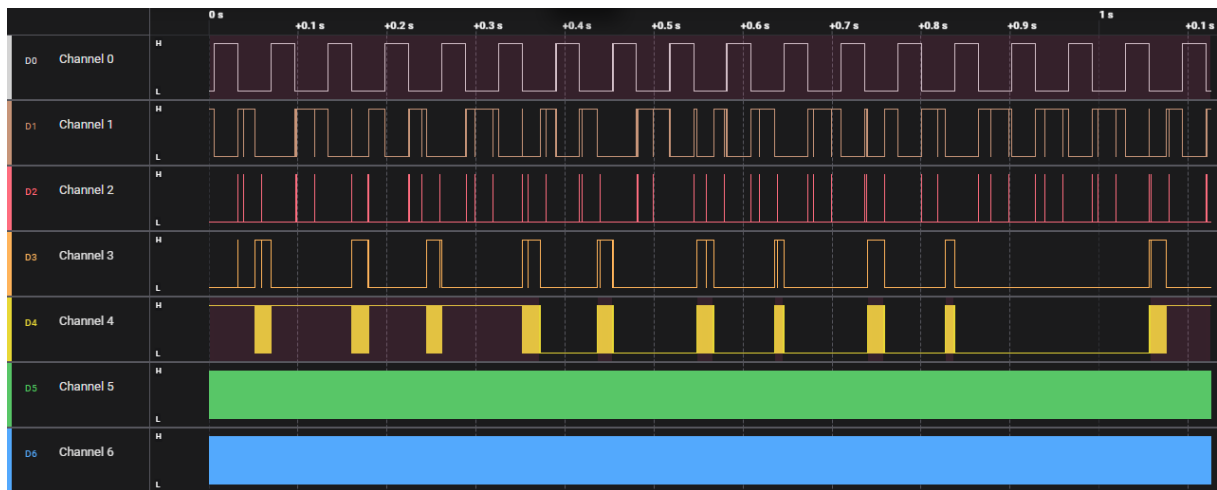


Figura 4: Capturas de ventana de 1seg



Figura 5: Tarea SD

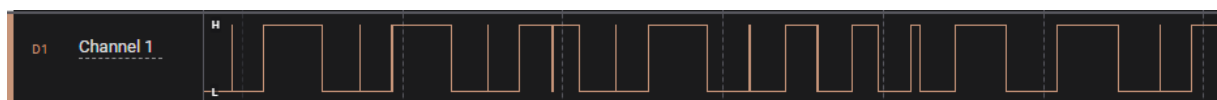


Figura 6: Tarea OLED



Figura 7: Tarea UI



Figura 8: Tarea Default



Figura 9: Tarea IDLE



Figura 10: Interrupción TIM3

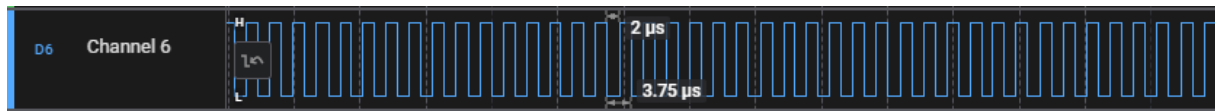


Figura 11: Salida PWM TIM2

Observaciones

- Frecuencia del PWM: **~280 kHz**
- Temporización de interrupciones: **cada 125 μs** (8 kHz)
- La tarea IDLE ocupa más tiempo antes de la reproducción y luego se reduce.
- La tarea SD es la que mayor prioridad tiene y se observa en la captura.
- La tarea OLED tiene un tiempo de ejecución considerable debido al uso de las funciones de la pantalla (ssd1306.c)

4.2. Factor de Utilización

A partir de las medidas realizadas desde el Analizador Lógico, se ha calculado el tiempo total que ocupa la tarea IDLE en una ventana de 1 segundo (Figura 4). Debido a que el factor de utilización mide todo lo que no es IDLE, al medir el porcentaje de tiempo que ejecuta la tarea IDLE, se obtiene el uso de CPU de la siguiente manera:

$$U_{total} = \frac{T_{total} - T_{idle}}{T_{total}} 100\% \quad (1)$$

Donde:

- T_{total} : tiempo total de captura
- T_{idle} : tiempo total en IDLE (Canal 4 en alto)
- $T_{total} - T_{idle}$: tiempo en uso (CPU ocupada).
- U : factor de utilización.

Por lo tanto se ha obtenido:

$$U_{total}[\%] = \frac{1,126 - 0,5196}{1,126} 100\% \quad (2)$$

$$U_{total}[\%] = 53,85\% \quad (3)$$

El valor de $U_{total} = 53,85\%$ indica que la CPU se encuentra ocupada poco más de la mitad del tiempo, dejando un margen considerable de tiempo en estado IDLE. Este porcentaje se encuentra dentro de la zona segura de operación, lo que asegura que el sistema posee capacidad de procesamiento suficiente para absorber picos de carga sin comprometer su desempeño o estabilidad.

5. Conclusiones

El sistema cumple todos los requerimientos del enunciado:

- Reproducción de audio con temporización a 8 kHz.
- Ecualización mediante filtro Biquad.
- Cálculo de FFT y visualización en pantalla OLED.
- Interfaz de usuario funcional con menú y selección.
- Sincronización correcta entre tareas e ISR usando semáforo y notificaciones
- Optimizado para funcionar en un microcontrolador de 64 KB FLASH y 20 KB RAM.